

吴恩达 NLP 课程

目录

- 吴恩达 NLP 课程 1
- 1. 情感分析 3
 - 1.1 逻辑回归处理 3
 - 1.2 朴素贝叶斯处理 3
 - 1.3 总结 6
 - 1.4 应用 7
 - 1.5 朴素贝叶斯的缺点 8
 - 1.6 误差分析 8
- 2. 词向量空间的背后思想 9
 - 2.1 向量空间模型 9
 - 2.2 距离 10
 - 2.3 PCA 12
- 3. 机器翻译 14
 - 3.1 转换矩阵 R 14
 - 3.2 局部敏感哈希 15
- 4. 单词自动纠错 17
 - 4.1 自动纠错步骤 17
 - 4.2 最小编辑距离 19
- 5 语音标记..... 21

5.1 语音标记简介	21
5.2 Markov 链.....	22
5.3 隐马尔科夫模型	23
5.4 转移矩阵和出度矩阵的计算	24
5.5 维特比算法 (Viterbi)	28
5.5.1 初始化 (第一列)	28
5.5.2 前向传递.....	29
5.5.3 后向传递.....	30

1. 情感分析

1.1 逻辑回归处理

见代码，较为简单

1.2 朴素贝叶斯处理

● 步骤一

要构建分类器，首先要创建下表给出的条件概率：

Positive tweets I am happy because I am learning NLP I am happy, not sad. Negative tweets I am sad, I am not learning NLP I am sad, not happy	word	Pos	Neg
	I	3	3
	am	3	3
	happy	2	1
	because	1	0
	learning	1	1
	NLP	1	1
	sad	1	2
	not	1	2
	N_{class}	13	12

这使我们可以计算以下概率表：

word	Pos	Neg
I	0.24	0.25
am	0.24	0.25
happy	0.15	0.08
because	0.08	0
learning	0.08	0.08
NLP	0.08	0.08
sad	0.08	0.17
not	0.08	0.17

一旦有了概率，就可以如下计算似然分数

Tweet: I am happy today; I am learning.

$$\prod_{i=1}^m \frac{P(w_i|pos)}{P(w_i|neg)} = \frac{0.14}{0.10} = 1.4 > 1$$

$$\frac{0.20}{0.20} * \frac{0.20}{0.20} * \frac{0.14}{0.10} * \frac{0.20}{0.20} * \frac{0.20}{0.20} * \frac{0.10}{0.10}$$

word	Pos	Neg
I	0.20	0.20
am	0.20	0.20
happy	0.14	0.10
because	0.10	0.05
learning	0.10	0.10
NLP	0.10	0.10
sad	0.10	0.15
not	0.10	0.15

分数大于 1 表示分类为正，否则为负。

● 步骤二:

加入 Laplace 平滑

Laplacian Smoothing

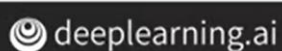
$$P(w_i|class) = \frac{\text{freq}(w_i, class)}{N_{class}}$$

class ∈ {Positive, Negative}

$$P(w_i|class) = \frac{\text{freq}(w_i, class) + 1}{N_{class} + V}$$

N_{class} = frequency of all words in class

V = number of unique words in vocabulary



you will add a new term in
the denominator V class.

修改步骤一中的概率计算，避免为 0

按 Esc 即可退出全屏模式

Introducing $P(w_i | \text{class})$ with smoothing

word	Pos	Neg
I	3	3
am	3	3
happy	2	1
because	1	0
learning	1	1
NLP	1	1
sad	1	2
not	1	2
Nclass	13	12

 $V = 8$

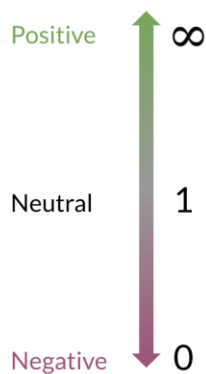
word	Pos	Neg
I	0.19	0.20
am	0.19	0.20
happy	0.14	0.10
because	0.10	0.05
learning	0.10	0.10
NLP	0.10	0.10
sad	0.10	0.15
not	0.10	0.15
Sum	1	1

longer has a probability of zero.

● 步骤三:

加入对数似然, 降低下溢风险。

要计算对数可能性, 我们需要获取比率并使用它们来计算分数, 这将使我们能够确定一条推文是正面的还是负面的。比率越高, 该词越积极:



word	Pos	Neg	ratio
I	0.19	0.20	
am	0.19	0.20	
happy	0.14	0.10	
because	0.10	0.05	
learning	0.10	0.10	
NLP	0.10	0.10	
sad	0.10	0.15	
not	0.10	0.15	

$$\text{ratio}(w_i) = \frac{P(w_i | \text{Pos})}{P(w_i | \text{Neg})}$$

$$\approx \frac{\text{freq}(w_i, 1) + 1}{\text{freq}(w_i, 0) + 1}$$

加入先验后的似然:

$$\frac{P(\text{pos})}{P(\text{neg})} \prod_{i=1}^m \frac{P(w_i | \text{pos})}{P(w_i | \text{neg})} > 1$$

第一个分量称为对数先验，第二个分量称为对数似然。

$$\log \left(\frac{P(pos)}{P(neg)} \prod_{i=1}^n \frac{P(w_i|pos)}{P(w_i|neg)} \right) \Rightarrow \log \frac{P(pos)}{P(neg)} + \sum_{i=1}^n \log \frac{P(w_i|pos)}{P(w_i|neg)}$$

doc: I am happy because I am learning.

$$\lambda(w) = \log \frac{P(w|pos)}{P(w|neg)}$$

$$\lambda(\text{happy}) = \log \frac{0.09}{0.01} \approx 2.2$$

word	Pos	Neg	λ
I	0.05	0.05	0
am	0.04	0.04	0
happy	0.09	0.01	
because	0.01	0.01	
learning	0.03	0.01	
NLP	0.02	0.02	
sad	0.01	0.09	
not	0.02	0.03	

doc: I am happy because I am learning.

$$\sum_{i=1}^m \log \frac{P(w_i|pos)}{P(w_i|neg)} = \sum_{i=1}^m \lambda(w_i)$$

$$\log \text{likelihood} = 0 + 0 + 2.2 + 0 + 0 + 0 + 1.1 = 3.3$$

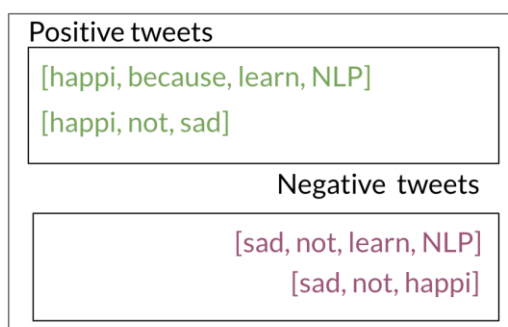
word	Pos	Neg	λ
I	0.05	0.05	0
am	0.04	0.04	0
happy	0.09	0.01	2.2
because	0.01	0.01	0
learning	0.03	0.01	1.1
NLP	0.02	0.02	0
sad	0.01	0.09	-2.2
not	0.02	0.03	-0.4

如上所见，3.3>0，我们会将文件分类为积极的。如果我们得到一个负数，我们将把它归为负数。（与之前不加对数的比值1相对应，比值大于1为正，小于1为负）

1.3 总结

- 得到推特上的数据集，既包含积极也包含消极的数据
- 对数据进行预处理
 - 小写化

- 移除标点符号，特殊字符，网址等
- 移除停用词
- 提取词干
- 对句子进行标记
- 计算词频



word	Pos	Neg
happi	2	1
because	1	0
learn	1	1
NLP	1	1
sad	1	2
not	1	2
N_{class}	7	7

freq(w, class)

- 分别计算不同类别的条件词频 $P(w | \text{pos}), P(w | \text{neg})$
- 计算对数似然比

$$\lambda(w) = \log \frac{P(w|\text{pos})}{P(w|\text{neg})}$$

- 计算对数先验，为对数似然比添加先验值

6) Compute $\log\text{prior} = \log(P(\text{pos})/P(\text{neg}))$

$\log\text{prior} = \log \frac{D_{\text{pos}}}{D_{\text{neg}}}$, where D_{pos} and D_{neg} correspond to the number of positive and negative documents respectively.

1.4 应用

朴素贝叶斯的许多应用包括：

- 作者识别
- 垃圾邮件过滤
- 信息检索
- 单词消歧

此方法通常用作简单的基准。它也真的很快。

1.5 朴素贝叶斯的缺点

第一，朴素贝叶斯（Naïve Bayes）进行条件独立性假设，而单词之间其实并非是完全独立的。
第二，训练集如果不均衡，训练出来的模型则对某一类会有很大的偏好，即并受语料库中单词频率的影响。

- 例如，如果您具有以下



“It is sunny and hot in the Sahara desert.”



“It's always cold and snowy in ___.”

在第一个图像中，您可以看到晴天和高温这两个词相互依赖，并且在一定程度上与“沙漠”一词相关。朴素贝叶斯自始至终都保持独立。此外，如果您要填写右边的句子，则该幼稚模型将为“春天，夏天，秋天，冬天”一词赋予相等的权重。

- 在 Twitter 上，正面推文通常多于负面推文。但是，您可能会发现某些“干净的”数据集是人为平衡的，具有相同数量的正向和负向推文。请记住，在现实世界中，数据可能要嘈杂得多。

1.6 误差分析

有一些错误可能导致您对示例或推文进行错误分类。例如，

- 删除标点符号
- 删除单词

Tweet: This is not good, because your attitude is not even close to being nice.

processed_tweet: [good, attitude, close, nice]

Tweet: My beloved grandmother :(

processed_tweet: [belov, grandmoth]

- 词序

Tweet: I am happy because I did not go.



Tweet: I am not happy because I did go.



- 对抗攻击

这些包括讽刺，讽刺，委婉语。

2. 词向量空间的背后思想

2.1 向量空间模型

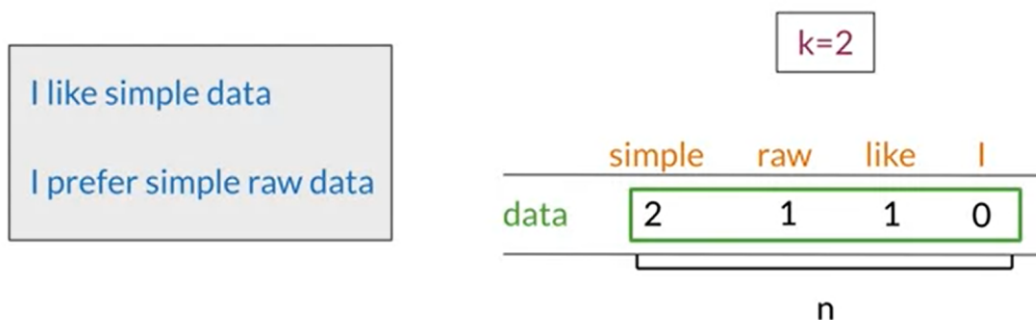
使用向量空间模型时，表示的方式是确定文本中每个单词周围的上下文，可用于识别以下内容的相似性 问题的回答，释义和总结。。概括地说，向量空间模型允许您将单词和文档表示为向量。这捕获了相对含义。

向量空间模型还将允许您捕获单词之间的依存关系。

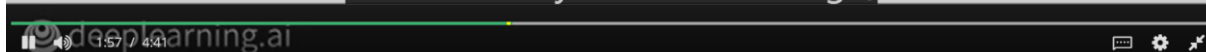
共生矩阵（超参数 K）

Word by Word Design

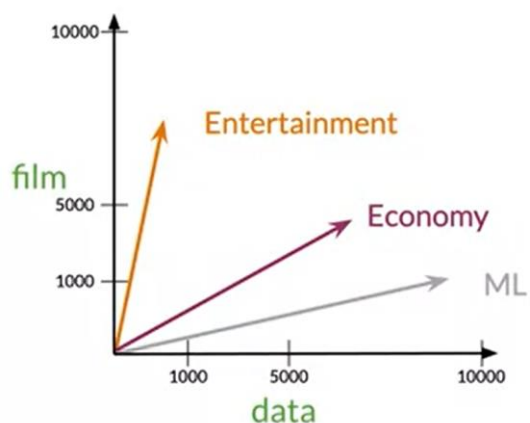
Number of times they occur together within a certain distance k



For a word by document design,



Vector Space

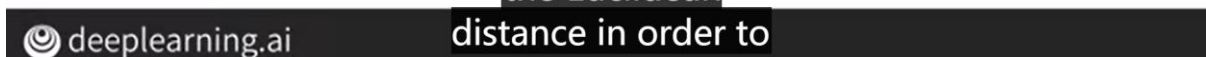


	Entertainment	Economy	ML
data	500	6620	9320
film	7000	4000	1000

Measures of "similarity:"
Angle
Distance

the Euclidean

distance in order to



2.2 距离

- 欧氏距离

Euclidean distance in Python

```
# Create numpy vectors v and w
v = np.array([1, 6, 8])
w = np.array([0, 4, 6])

# Calculate the Euclidean distance d
d = np.linalg.norm(v-w)
# Print the result
print("The Euclidean distance between v and w is: ", d)
```

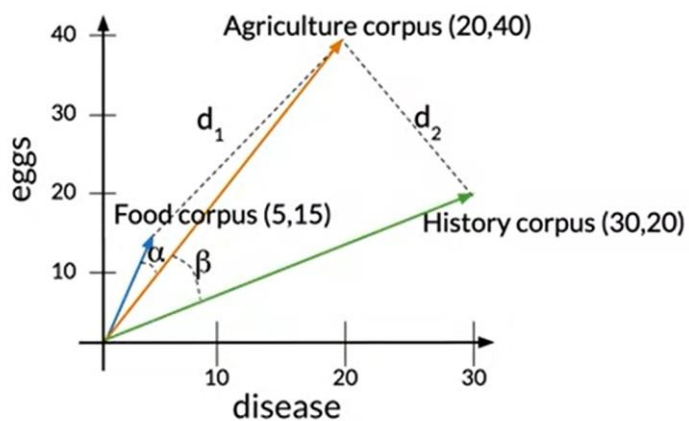
The Euclidean distance between v and w is: 3

the norm function works for

n-dimensional spaces.

- 余弦相似度

Euclidean distance vs Cosine similarity

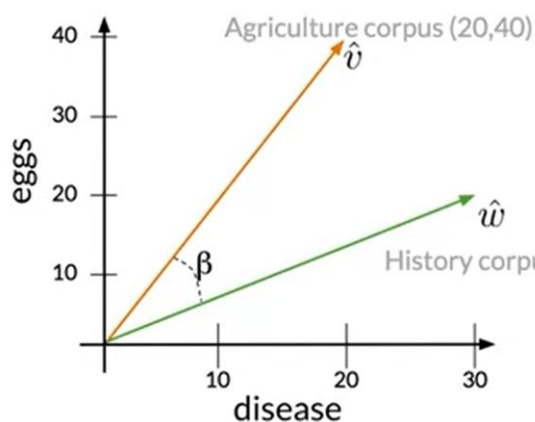


vector representations than

their euclidean distance.

请记住，此度量标准在欧几里德距离上的主要优势是它不受制于表示之间的大小差异。重要的一点是，该指标是成比例的要比较的向量的方向之间的相似性。

Cosine Similarity



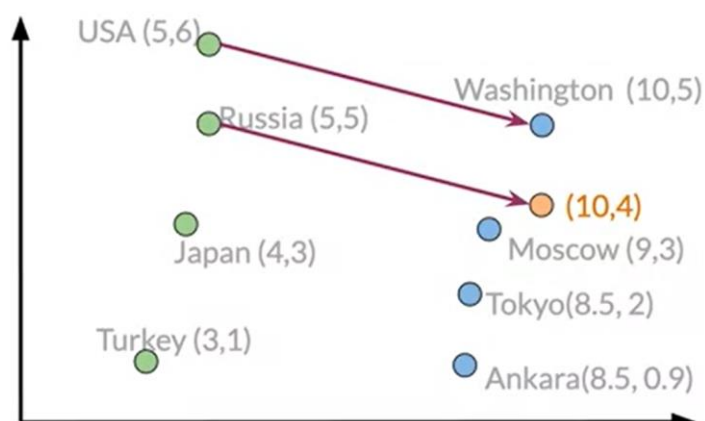
$$\hat{v} \cdot \hat{w} = \|\hat{v}\| \|\hat{w}\| \cos(\beta)$$

$$\cos(\beta) = \frac{\hat{v} \cdot \hat{w}}{\|\hat{v}\| \|\hat{w}\|}$$

$$= \frac{(20 \times 30) + (40 \times 20)}{\sqrt{20^2 + 40^2} \times \sqrt{30^2 + 20^2}}$$

Ultimately, you should get
a cosine similarity of 0.87.

Manipulating word vectors



$$\text{Washington} - \text{USA} = \begin{bmatrix} 5 & -1 \end{bmatrix}$$

$$\text{Russia} + \begin{bmatrix} 5 & -1 \end{bmatrix} = \begin{bmatrix} 10 & 4 \end{bmatrix}$$

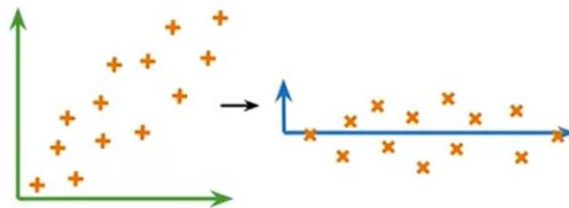
[Mikolov et al, 2013, Distributed Representations of Words and Phrases and their Compositionality]

At the end, you should
deduce that the capital of

2.3 PCA

PCA 是用于降维可以为您的数据找到互不相关的特征。对于可视化数据非常有帮助，检查您的表示是否正在捕获单词之间的关系。

PCA algorithm

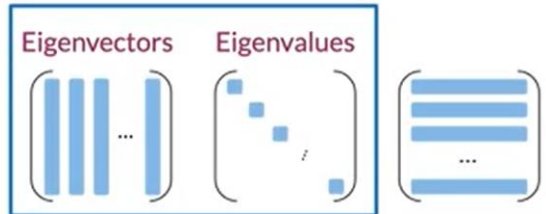


按 Esc 即可退出全屏模式

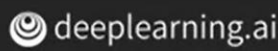
Mean Normalize Data $x_i = \frac{x_i - \mu_{x_i}}{\sigma_{x_i}}$

Get Covariance Matrix Σ

Perform SVD $SVD(\Sigma)$



The first of those matrices contain

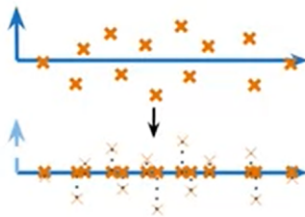


the eigenvector stacked column wise.

分享

PCA algorithm

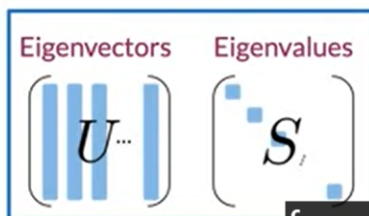
按 Esc 即可退出全屏模式



Dot Product to Project Data $X' = XU[:, 0:2]$

Percentage of Retained Variance

$$\frac{\sum_{i=0}^1 S_{ii}}{\sum_{j=0}^d S_{jj}}$$



from your original embedding.



PCA 的步骤总结:

Summary

- Eigenvectors give the direction of uncorrelated features
- Eigenvalues are the variance of the new features
- Dot product gives the projection on uncorrelated features

Good luck with that.



3. 机器翻译

3.1 转换矩阵 R

Align word vectors

$$\begin{pmatrix} \text{["cat" vector]} \\ \text{[... vector]} \\ \text{["zebra" vector]} \end{pmatrix} \mathbf{X} \mathbf{R} \approx \mathbf{Y} \begin{pmatrix} \text{["chat" vecteur]} \\ \text{[... vecteur]} \\ \text{["zébresse" vecteur]} \end{pmatrix} \mathbf{Y}$$

subsets of the full vocabulary

And if it works well, then the model can be used to translate words that are not

Solving for R

initialize R

in a loop:

$$Loss = \| \mathbf{X}\mathbf{R} - \mathbf{Y} \|_F$$

$$g = \frac{d}{dR} Loss \quad \text{gradient}$$

$$R = R - \alpha g \quad \text{update}$$

break out of the loop when the loss
falls between a certain threshold.



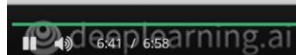
分享

Gradient

$$Loss = \| \mathbf{X}\mathbf{R} - \mathbf{Y} \|_F^2$$

$$g = \frac{d}{dR} Loss = \frac{2}{m} (\mathbf{X}^T (\mathbf{X}\mathbf{R} - \mathbf{Y}))$$

It's easier to take



the derivative of this expression

3.2 局部敏感哈希

总之一句话，LSH 不像树形结构的方法可以得到精确的结果，LSH 所得到的是一个近似的结果，因为在很多领域中并不需非常高的精确度。即使是近似解，但有时候这个近似程度几乎和精准解一致。

LSH 的主要思想是，高维空间的两点若距离很近，那么设计一种哈希函数对这两点进行哈希值计算，使得他们哈希值有很大的概率是一样的。同时若两点之间的距离

较远，他们哈希值相同的概率会很小。

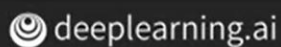
看这篇博客：

<https://www.cnblogs.com/wt869054461/p/9234184.html>

哈希值的计算方法：

Multiple planes, single hash value!!

```
def hash_multiple_plane(P_l,v):
    hash_value = 0
    for i, P in enumerate(P_l):
        sign = side_of_plane(P,v)
        hash_i = 1 if sign >=0 else 0
        hash_value += 2**i * hash_i
    return hash_value
```



分割平面而对设计思路：

Make one set of random planes

```
num_dimensions = 2 #300 in assignment
num_planes = 3 #10 in assignment
random_planes_matrix = np.random.normal(
    size=(num_planes,
          num_dimensions))

array([[ 1.76405235  0.40015721]
       [ 0.97873798  2.2408932 ]
       [ 1.86755799 -0.97727788]])

v = np.array([[2,2]])
```

```
def side_of_plane_matrix(P,v):
    dotproduct = np.dot(P,v.T)
    sign_of_dot_product = np.sign(dotproduct)
    return sign_of_dot_product

num_planes_matrix = side_of_plane_matrix(
    random_planes_matrix,v)

array([[1.]
       [1.]
       [1.]])
```

See notebook for calculating the hash value!

but please, do check out
the lecture notebook



计算文档的向量

Document vectors

```
word_embedding = {"I": np.array([1,0,1]),  
                  "love": np.array([-1,0,1]),  
                  "learning": np.array([1,0,1])}  
words_in_document = ['I', 'love', 'learning']  
document_embedding = np.array([0,0,0])  
  
for word in words_in_document:  
    document_embedding += word_embedding.get(word,0)  
  
print(document_embedding)
```

You add these all up and return

the documents embedding.

4. 单词自动纠错

4.1 自动纠错步骤

自动校正的工作原理。有四个关键步骤。

首先，找出不正确的词，拼写错误是执行此操作的一种方法。（对比语料库中是否存在这个单词）

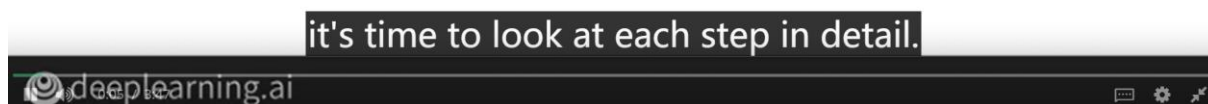
二，找到字符串 1、2 3，或任何 n 个编辑距离。别担心，您会学到的 即将达到最小编辑距离。现在，重要的是要知道 是距离您输入的字符串一个编辑距离，与之相比，它更类似于您的字符串 一个距离两个编辑距离的字符串。

第三，过滤字符串 拼写正确的真实单词。（仍旧是对比语料库）

第四，计算单词概率，告诉您每个单词出现在其中的可能性 在这种情况下，选择 最有可能的候选人是替代人。

Building the model

1. Identify a misspelled word
2. Find strings n edit distance away
3. Filter candidates
4. Calculate word probabilities



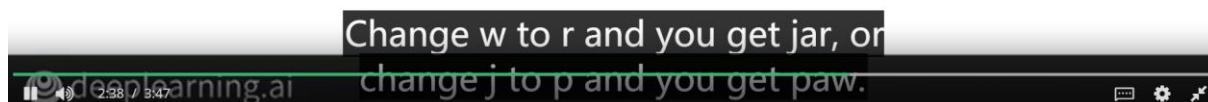
第二个步骤的详解：

Building the model

按 Esc 即可退出全屏模式

2. Find strings n edit distance away

- Edit: an operation performed on a string to change it
- Insert (add a letter) 'to': 'top', 'two' ...
- Delete (remove a letter) 'hat': 'ha', 'at', 'ht'
- Switch (swap 2 adjacent letters) 'eta': 'eat', 'tea'
- Replace (change 1 letter to another) 'jaw': 'jar', 'paw', ...



四 计算单词概率。最后一步是计算 单词概率和查找 候选人中最有可能出现的单词。

任何单词的概率 语料库中的是 单词出现两次的次数 按总字数 例如，单词“m” 出现两次，语料库的大小为七个。对于自动更正，您会找到带有 最高机率并选择 这个词作为替代，仅此而已。

Summary

1. Identify a misspelled word
2. Find strings n edit distance away

Insert
Delete
Switch
Replace

3. Filter candidates
4. Calculate word probabilities

$$P(w) = \frac{C(w)}{V}$$

deah → dear ✓
yeah
dear
dean
... etc

strings, and many more.

deeplearning.ai

4.2 最小编辑距离

和动态规划是一样的想法，将最问题不断地拆解为更小的子问题。

Minimum edit distance

- Example:

Source:

p l a y



Target:

s t a y

Edit cost:

Insert 1
Delete 1
Replace 2

edit distance = 2 * 2 = 4

p → s : replace
l → t : replace

} edits = 2

the minimum edit distance

just by looking at it.

deeplearning.ai

Minimum edit distance

Source: play → Target: stay

Cost: insert: 1, delete: 1, replace: 2

p → s

insert + delete: p → ps → s: 2

delete + insert: p → # → s: 2

replace: p → s: 2

	0	1	2	3	4
	#	s			
0	#	0	1		
1	p	1	2		
2					
3					
4					

you will notice that

to fill in a cell,



分享

按 Esc 即可退出全屏模式

Minimum edit distance

Source: play → Target: stay

Cost: insert: 1, delete: 1, replace: 2

p → s

$$D[i, j] = \min \begin{cases} D[i-1, j] + del_cost \\ D[i, j-1] + ins_cost \\ D[i-1, j-1] + \begin{cases} rep_cost; & \text{if } src[i] \neq tar[j] \\ 0; & \text{if } src[i] = tar[j] \end{cases} \end{cases}$$

		0	1	2	3	4
		#	s	t	a	y
0	#	0	1	2	3	4
1	p	1	2			
2	l	2				
3	a	3				
4	y	4				

because there is

no edit to be done



2:39 / 3:54

按 Esc 即可退出全屏模式

Minimum edit distance

Source: play → Target: stay

Cost: insert: 1, delete: 1, replace: 2

play → stay

$D[m, n] = 4$

	0	1	2	3	4	
	#	s	t	a	y	
0	#	0	1	2	3	4
1	p	1	2	3	4	5
2	l	2	3	4	5	6
3	a	3	4	5	4	5
4	y	4	5	6	5	4

Can you see what's happened from

5 语音标记

5.1 语音标记简介

语音标记指的是为文本，单词或者语音打上标签。例如，我们可以对任意一句英语文本中的每个单词标记词性，如名词，动词，介词，w-介词等。再如命名实体，长城在北京这句话中，长城和北京都被打上实体的标签。或者在语音识别的领域使用语音标签检查某一些单词是否具有较高概率。

Part of speech (POS) tagging

Part of speech tags:

lexical term	tag	example
noun	NN	something, nothing
verb	VB	learn, study
determiner	DT	the, a
w-adverb	WRB	why, where
...	...	

Why not learn something?

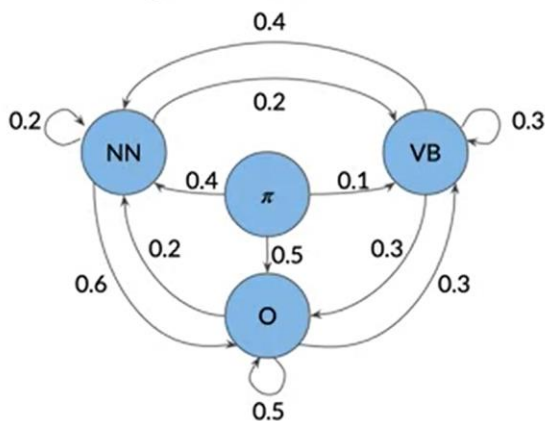
WRB RB VB NN .

or POS tagging for short,

deeplearning.ai

5.2 Markov 链

Initial probabilities



$A =$

	NN	VB	O
π (initial)	0.4	0.1	0.5
NN (noun)	0.2	0.2	0.6
VB (verb)	0.4	0.3	0.3
O (other)	0.2	0.3	0.5

So now it has dimensions n plus 1 by n .

deeplearning.ai

Transition table and matrix

$$A =$$

	NN	VB	O
π (initial)	0.4	0.1	0.5
NN (noun)	0.2	0.2	0.6
VB (verb)	0.4	0.3	0.3
O (other)	0.2	0.3	0.5

$$A = \begin{pmatrix} 0.4 & 0.1 & 0.5 \\ 0.2 & 0.2 & 0.6 \\ 0.4 & 0.3 & 0.3 \\ 0.2 & 0.3 & 0.5 \end{pmatrix}$$

Summary

States

$$Q = \{q_1, \dots, q_N\}$$

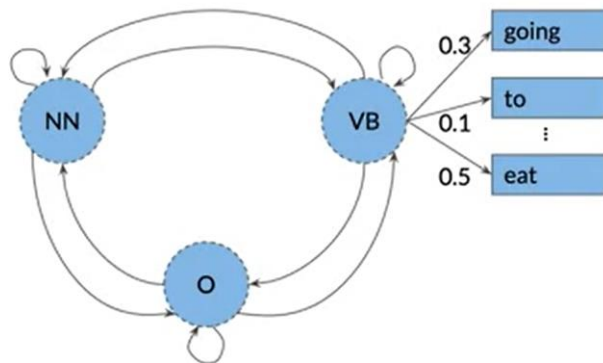
Transition matrix

$$A = \begin{pmatrix} a_{1,1} & \dots & a_{1,N} \\ \vdots & \ddots & \vdots \\ a_{N+1,1} & \dots & a_{N+1,N} \end{pmatrix}$$

5.3 隐马尔科夫模型

按 Esc 即可退出全屏模式

Emission probabilities


 $B =$

	going	to	eat	...
NN (noun)	0.5	0.1	0.02	
VB (verb)	0.3	0.1	0.5	
O (other)	0.3	0.5	0.68	

Each row is designated for one of the hidden states.

Summary

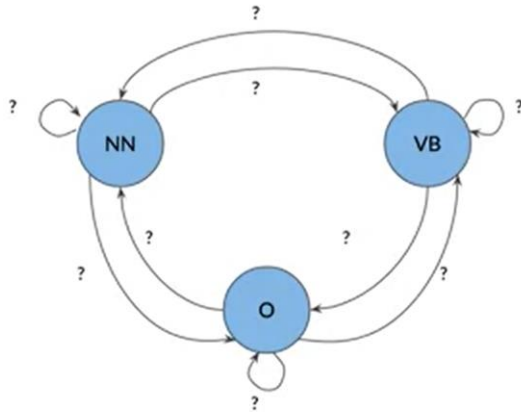
States	Transition matrix	Emission matrix
$Q = \{q_1, \dots, q_N\}$	$A = \begin{pmatrix} a_{1,1} & \dots & a_{1,N} \\ \vdots & \ddots & \vdots \\ a_{N+1,1} & \dots & a_{N+1,N} \end{pmatrix}$	$B = \begin{pmatrix} b_{11} & \dots & b_{1V} \\ \vdots & \ddots & \vdots \\ b_{N1} & \dots & b_{NV} \end{pmatrix}$

出度矩阵表示您过渡的概率 n 个隐藏状态代表您的语料库中 n 个单词的语音标记。第一张图中的概率行和并不是 1，这不矛盾。因为即便是同一个单词，其词性和含义也不太一样，例如 back，既有后背的意思，也有回来，后面的这些意思。

5.4 转移矩阵和出度矩阵的计算

转移概率公式：

Transition probabilities



1. Count occurrences of tag pairs

$$C(t_{i-1}, t_i)$$

2. Calculate probabilities using the counts

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i)}{\sum_{j=1}^N C(t_{i-1}, t_j)}$$

Populating the transition matrix

$A =$

	NN	VB	O
π	1		
NN (noun)	0		
VB (verb)	0		
O (other)	6		

<s> in a station of the metro
 <s> the apparition of these faces in the crowd :
 <s> petals on a wet , black bough .

Ezra Pound - 1913

A noun following and other
 tag O occurs six times.

Populating the transition matrix

$$A =$$

	NN	VB	O
π	1	0	2
NN (noun)	0	0	6
VB (verb)	0	0	0
O (other)	6	0	8

<s> in a station of the metro
 <s> the apparition of these faces in the crowd :
 <s> petals on a wet, black bough .

Ezra Pound - 1913

on a a wet wet,

deeplearning.ai

Smoothing

$$A =$$

	NN	VB	O	
π	$1+\epsilon$	$0+\epsilon$	$2+\epsilon$	$3+3*\epsilon$
NN	$0+\epsilon$	$0+\epsilon$	$6+\epsilon$	$6+3*\epsilon$
VB	$0+\epsilon$	$0+\epsilon$	$0+\epsilon$	$0+3*\epsilon$
O	$6+\epsilon$	$0+\epsilon$	$8+\epsilon$	$14+3*\epsilon$

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i) + \epsilon}{\sum_{j=1}^N C(t_{i-1}, t_j) + N * \epsilon}$$

a small value epsilon

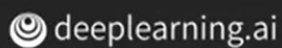
deeplearning.ai

Smoothing

$A =$

	NN	VB	O
π	0.3333	0.0003	0.6663
NN	0.0001	0.0001	0.9996
VB	0.3333	0.3333	0.3333
O	0.4285	0.0000	0.5713

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i) + \epsilon}{\sum_{j=1}^N C(t_{i-1}, t_j) + N * \epsilon}$$



that you no longer have any
zero value with entries in A.

实际示例中，您可能不想将平滑应用于初始概率，即转换矩阵的第一行。那是因为如果您对那一行应用平滑，向可能为零的条目添加一个小值，您将有效地允许一个句子开头语音标签的任何部分，包括标点符号。

The emission matrix

$B =$

	in	a	...
NN (noun)	0		
VB (verb)	0		
O (other)	2		

<s> in a station of the metro
<s> the apparition of these faces in the crowd :
<s> petals on a wet , black bough .

Ezra Pound - 1913



associated two times with
the word "in" in the corpus.

5.5 维特比算法 (Viterbi)

Viterbi algorithm – Steps

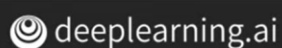
1. Initialization step
2. Forward pass
3. Backward pass

$$C =$$

	w_1	w_2	...	w_K
t_1				
...				
t_N				

$$D =$$

	w_1	w_2	...	w_K
t_1				
...				
t_N				

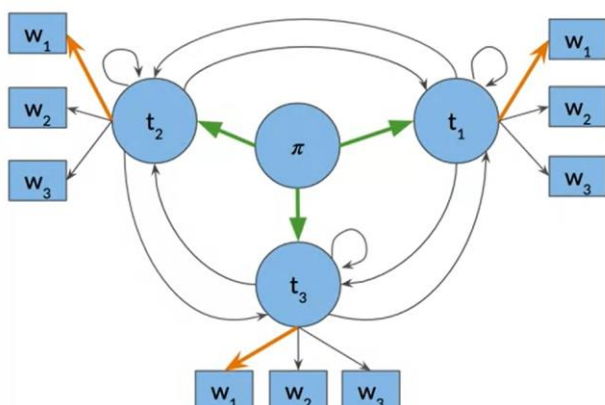


主要是熟悉一句话的概率是怎么计算得来的。

5.5.1 初始化 (第一列)

C 矩阵很好理解，记录的是相邻单词之间的转移概率。

Initialization step



$$C =$$

	w_1	w_2	...	w_K
t_1	$c_{1,1}$			
...				
t_N	$c_{N,1}$			

$$c_{i,1} = \pi_i * b_{i, \text{cindex}(w_1)}$$

$$= a_{1,i} * b_{i, \text{cindex}(w_1)}$$

for the given word here, W1.

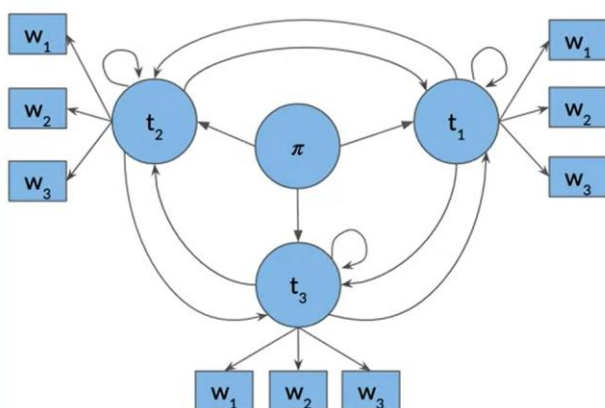


在 D 矩阵中，当你找到给定单词序列的词性标签的最有可能的序列时，你存储了代表你正在

遍历的不同状态的标签, w_1 一直到 w_k 。

In the D matrix, you store the labels that represent the different states you're traversing when finding the most likely sequence of parts of speech tags for the given sequence of words, w_1 all the way to w_k .

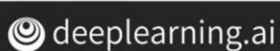
Initialization step



$$D = \begin{array}{c|cccc} & w_1 & w_2 & \dots & w_K \\ \hline t_1 & d_{1,1} & & & \\ \dots & & & & \\ t_N & d_{N,1} & & & \end{array}$$

$$d_{i,1} = 0$$

In the first column, you simply

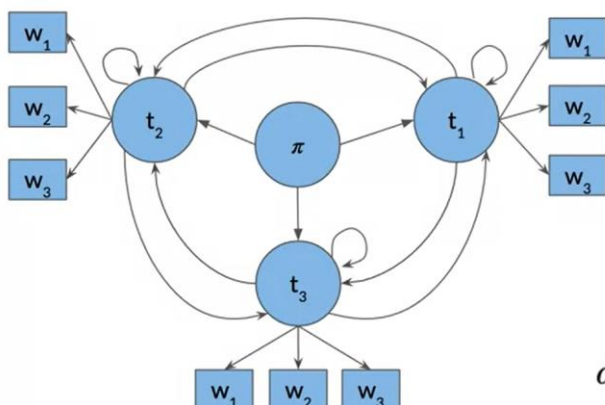


set all entries to zero,

5.5.2 前向传递

分享

Forward pass

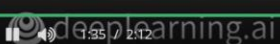


$$D = \begin{array}{c|cccc} & w_1 & w_2 & \dots & w_K \\ \hline t_1 & d_{1,1} & d_{1,2} & & d_{1,K} \\ \dots & & & & \\ t_N & d_{N,1} & d_{N,2} & & d_{N,K} \end{array}$$

$$c_{i,j} = \max_k c_{k,j-1} * a_{k,i} * b_{i, \text{index}(w_j)}$$

$$d_{i,j} = \arg\max_k c_{k,j-1} * a_{k,i} * b_{i, \text{index}(w_j)}$$

that are not the initial state,



5.5.3 后向传递

Backward pass

$$C = \begin{array}{c|ccccc} & w_1 & w_2 & \dots & w_K \\ \hline t_1 & c_{1,1} & c_{1,2} & & c_{1,K} \\ \hline \dots & & & & \\ \hline t_N & c_{N,1} & c_{N,2} & & c_{N,K} \end{array} \quad D = \begin{array}{c|ccccc} & w_1 & w_2 & \dots & w_K \\ \hline t_1 & d_{1,1} & d_{1,2} & & d_{1,K} \\ \hline \dots & & & & \\ \hline t_N & d_{N,1} & d_{N,2} & & d_{N,K} \end{array}$$

$$s = \operatorname{argmax}_i c_{i,K}$$

Backward pass

$$C = \begin{array}{c|ccccc} & w_1 & w_2 & w_3 & w_4 & w_5 \\ \hline t_1 & 0.25 & 0.125 & 0.025 & 0.0125 & 0.01 \\ \hline t_2 & 0.1 & 0.025 & 0.05 & 0.01 & 0.003 \\ \hline t_3 & 0.3 & 0.05 & 0.025 & 0.02 & 0.0000 \\ \hline t_4 & 0.2 & 0.1 & 0.000 & 0.0025 & 0.0003 \end{array}$$

$$s = \operatorname{argmax}_i c_{i,K} = 1$$

Backward pass

$$D =$$

	w_1	w_2	w_3	w_4	w_5
t_1	0	1	3	2	3
t_2	0	2	4	1	3
t_3	0	2	4	1	4
t_4	0	4	4	3	1

$$s = \underset{i}{\operatorname{argmax}} c_{i,K} = 1$$

<s> w1 w2 w3 w4 w5

So the most likely states of word

w5 is the t_1 parts of speech tag.

Backward pass

$$D =$$

	w_1	w_2	w_3	w_4	w_5
t_1	0	1	3	2	3
t_2	0	2	4	1	3
t_3	0	2	4	1	4
t_4	0	4	4	3	1

<s> w1 w2 w3 w4 w5

$\pi \leftarrow t_2 \leftarrow t_3 \leftarrow t_1 \leftarrow t_3 \leftarrow t_1$

Implementation notes

1. In Python index starts with 0!
2. Use log probabilities

$$c_{i,j} = \max_k c_{k,j-1} * a_{k,i} * b_{i, \text{cindex}(w_j)}$$



$$\log(c_{i,j}) = \max_k \log(c_{k,j-1}) + \log(a_{k,i}) + \log(b_{i, \text{cindex}(w_j)})$$

use log probabilities instead, where

deep learning numbers are summed instead of multiplied.